

Field Notes from a Provincial System

Errors, crashes and silent failures in a multi-platform library network – and how each one was found

Drawn from the development of NEXLIB, a library management system for Government Degree Colleges under the Higher Education Department, Khyber Pakhtunkhwa.

Four applications · ~57,700 lines · Android, Windows, and two web portals.

Preface: why these notes exist

Most books about debugging are written backwards. The author knows the answer before the first sentence, so the bug arrives already labelled, the investigation runs in a straight line, and the reader never experiences the thing that actually makes debugging hard: not knowing, for hours, which of forty possibilities is true.

This book is assembled from a real project's wreckage. Every failure in it happened, in order, to a working system with real users and real library data. The diagnoses are the ones that were actually reached – including the ones that were wrong first, and had to be abandoned.

NEXLIB is a library management system for the government degree colleges of Khyber Pakhtunkhwa. It is four separate applications – an Android app, a Windows desktop program, a college web portal and a directorate portal – sharing one multi-tenant cloud database, and designed to keep working in colleges where the internet does not. That shape generates a particular and instructive class of bug: faults that appear on one platform and not another, faults that are invisible until a second device disagrees, and faults that never crash anything at all.

Three convictions run through what follows.

A crash is the best kind of failure. It is loud, it names a file and a line, and it stops before doing more damage. The failures that cost this project the most time never crashed: a delete that quietly did nothing, a status badge that reported the wrong problem, a wipe that reported success while leaving most of the data behind. Chapter after chapter here is about failures that looked like nothing.

The error message is usually not the error. `Connection to the Kotlin daemon has been unexpectedly lost` is an out-of-memory condition. `INVALID_LOGIN_CREDENTIALS` frequently means the account does not exist. A library book reappearing after deletion is a query that excluded the wrong rows. Learning to read past the message to the mechanism is most of the skill.

Architecture decides which bugs are possible. The desktop app talks to the database through an administrative credential that bypasses every security rule. That one decision made an entire category of fault invisible on the platform the developer used daily, and guaranteed it would appear only on the platforms he could not easily test. No amount of care compensates for that; only changing the architecture does.

The chapters are grouped so they can be read out of order. Each case study follows the same shape: the symptom as it was first seen, the false trails, the mechanism, the fix, and what it should have taught us earlier.

Contents

Chapters marked *written* are complete in this edition. The remainder are outlined and drafted in sequence.

PART I · THE SYSTEM AND ITS SHAPE

1 Four applications, one database

OUTLINED

2	What multi-tenancy actually requires	OUTLINED
3	Offline-first: the inversion that changes everything	OUTLINED
PART II · CRASHES		
4	What a crash actually is	WRITTEN
5	Two crashes, dissected	WRITTEN
6	The crash that was not a crash	WRITTEN
7	Reading a traceback you did not write	OUTLINED
8	Crashes that only happen after packaging	OUTLINED
PART III · FAILURES THAT NEVER CRASH		
9	The cost of catching everything	WRITTEN
10	The delete that came back	WRITTEN
11	The status badge that blamed the network	WRITTEN
12	Phantoms: deleting what is already gone	WRITTEN
13	Last-write-wins and the clock you do not control	WRITTEN
PART IV · SECURITY FAILURES		
14	One rule, every college's data	WRITTEN
15	The button that made everyone an administrator	OUTLINED
16	Shipping a key that unlocks everything	OUTLINED
17	Testing rules instead of trusting them	OUTLINED
PART V · SCALE, COST AND MEMORY		
18	Read amplification: the 155-fold bill	OUTLINED
19	Why the catalogue size was bounded by RAM	OUTLINED
20	Measuring instead of projecting	OUTLINED
PART VI · BUILD AND DELIVERY		
21	Builds that fail for reasons unrelated to code	OUTLINED
22	Wiping a system so it stays wiped	OUTLINED
23	The delivery checklist	OUTLINED
PART VII · PRACTICE		
24	Building a diagnostic tool	OUTLINED

25	Instrumenting a system you cannot visit	OUTLINED
26	What we would do differently	OUTLINED

PART II

Crashes

The failures that announce themselves – and are therefore the easiest kind to have.

CHAPTER 4

What a crash actually is

Before dissecting particular crashes it is worth being precise about what one is, because the word covers at least four different events that need different responses.

In a Python desktop application, a crash is usually an *unhandled exception*: some function raised, no enclosing block caught it, and the interpreter unwound the stack to the top and stopped. In a Kotlin Android application it is much the same, with the operating system killing the process at the end. But "the app crashed" is also said about a process killed by the operating system for using too much memory, about a window that stopped repainting, and about a program that exited cleanly for reasons the user did not understand. These have almost nothing in common except the user's experience of them.

The distinction matters because only the first leaves evidence.

The global handler, and why it is not optional

A GUI application without a top-level exception handler simply vanishes. The user sees the window disappear; there is no message, no log, nothing to report. The desktop half of NEXLIB installs one at startup:

```
def _global_exception_handler(exc_type, exc_value, exc_tb):
    """Show a dialog for unhandled exceptions instead of silently crashing."""
    msg = "".join(traceback.format_exception(exc_type, exc_value, exc_tb))
    logger.critical(f"Unhandled exception: {msg}")
    try:
        dlg = QMessageBox()
        dlg.setWindowTitle("Unexpected Error")
        dlg.setText("An unexpected error occurred. The application may be unstable.")
        dlg.setDetailedText(msg)
        dlg.setIcon(QMessageBox.Icon.Critical)
        dlg.exec()
    except Exception:
        pass

sys.excepthook = _global_exception_handler
```

gdc_desktop/main.py

Three things in that handler are worth noting, because each was learned rather than designed.

It **logs before it displays**. Showing a dialog requires the Qt event loop to be in a sane state, and the thing that just crashed may have left it otherwise. Writing to the log first means the evidence survives even when the dialog cannot be drawn.

It puts the traceback in `setDetailedText` rather than the main message. A librarian sees one plain sentence; the person diagnosing it clicks "Show Details" and gets the whole stack. A dialog that dumps a traceback at a non-technical user teaches them to dismiss dialogs without reading.

It **wraps its own dialog in a try/except**. A crash handler that crashes produces the original silent disappearance plus a confusing second failure.

The handler that does not catch everything

Here is the limitation that cost this project real time. `sys.excepthook` is called for unhandled exceptions *on the main thread*. An exception raised inside a `QThread` worker never reaches it.

NEXLIB has twenty-two files that use `QThread`, and background workers are precisely where the risky work lives: network calls, Firestore syncs, report generation, bulk imports. The application's most failure-prone code was the part the crash handler could not see. A sync worker could raise, die, and leave the user with a library that simply stopped synchronising – no dialog, no log entry, nothing.

The fix is a second handler, introduced in Python 3.8:

```
threading.excepthook = _thread_exception_handler
```

Any platform with background threads has this same hole. Android needs

`Thread.setDefaultUncaughtExceptionHandler`; JavaScript needs both `window.onerror` and a separate `unhandledrejection` listener, because a rejected promise is not an error event and the two are reported through different channels.

LESSON

Count your entry points for failure, not just your handlers. Every thread, every event loop, every promise chain is a place an exception can escape, and each needs its own catch of last resort.

What a crash log is for

A log that nobody reads is not instrumentation; it is a file. The desktop app wrote crashes faithfully to `gdc_library.log` for months, and in that time nobody opened it once – not from negligence, but because a log is an undifferentiated stream and the interesting part is buried.

What made the log useful was grouping. One bug that fires forty times produces forty tracebacks, which reads as catastrophe; grouped by fault it reads as one problem with a count, which is what it is:

```
CRASHES – 2 total, 1 distinct
```

```
2 x ValueError: Invalid image mode
```

```
ERRORS – 2 total, 1 distinct
```

```
2 x Push failed: DEADLINE_EXCEEDED
```

```
first 2026-09-20 09:30:00, last 2026-09-20 11:30:00
```

Two numbers do most of the work here. *Distinct* tells you how many problems you have. *Total* tells you how much they are hurting. A fault with a count of two and a fault with a count of four hundred deserve very different attention, and the raw log makes that distinction almost impossible to see.

There is a subtlety in the grouping worth recording, because getting it wrong made the first version of that report useless. The global handler writes a line beginning `Unhandled exception:` followed by the traceback. A naive error tally counts that line too — so every crash appeared twice: once under its real fault, and once under the generic heading that names nothing. The tally has to skip the line that opens a traceback, because it is already counted by the fault it introduces.

The chapters that follow examine specific crashes. Keep two questions in mind for each: *what did the user actually see?* and *what evidence survived?* In nearly every case the gap between those two answers is where the time went.

CHAPTER 5

Two crashes, dissected

These two arrived on the same afternoon, in the same application, and looked identical to the user: a dialog saying "Unexpected Error". Their causes had nothing in common, and the difference between them is instructive.

Case one: a variable that was never defined

Symptom. A librarian clicks "Generate Full Analytics PDF". The dialog appears. No report is produced.

Evidence. The detailed text showed:

```
NameError: name 'gold' is not defined
  File "services/report_service.py", line 333, in generate_full_analytics_report
```

Mechanism. Line 333 used a colour variable named `gold` to draw a horizontal rule. Two other report functions in the same file defined `gold` as a local at the top of their bodies. This one did not. The function had been written by copying a neighbour and editing the middle, and the definition lived above the copied region.

This is as simple as a bug gets, and it survived into a shipped application for a reason worth naming: **Python does not check that a name exists until the line runs**. Any function whose unhappy path is never exercised can carry a `NameError` indefinitely. The report generated correctly for months because line 333 sat inside a branch that only executed when the analytics section had data to draw a rule under.

Fix. Define the local, as its neighbours do.

The interesting question is not how to fix it but how it was allowed to exist. A linter would have caught this in under a second – `pyflakes` reports undefined names without running anything. The project had no linter and no continuous integration. One tool, configured once, would have found this class of fault across all 20,000 lines of Python.

LESSON

In a dynamically typed language, unexecuted code is unverified code. Static analysis is not a luxury; it is the only thing standing between you and a `NameError` in a branch nobody has taken yet.

Case two: an image in the wrong mode

Symptom. Opening the Digital ID Cards screen produced the same "Unexpected Error" dialog. The screen never appeared at all.

Evidence. The traceback ended inside Qt's image conversion, called from a function that turns a member's ID into a QR code.

The false trail. The obvious first hypothesis was a bad member record – a null field, an unprintable character in a name. That hypothesis is attractive because it is the kind of bug one expects, and it wasted time. The crash occurred for every member, including ones whose records were plainly clean.

Mechanism. The `qrcode` library returns a PIL image in mode `"1"` – one bit per pixel, the natural representation for something that is only ever black or white. Qt's `ImageQt` bridge accepts several PIL modes, but support for 1-bit images varies across combinations of Pillow and Qt versions. On this machine's combination, it did not.

So the code was correct, the data was correct, and the crash was real. What failed was an assumption about a dependency's behaviour that had held on the developer's previous environment and stopped holding on this one.

Fix. Convert to a mode every version supports before handing the image to Qt:

```
# qrcode returns a 1-bit ("1" mode) image, which PIL's ImageQt does not
# support on all Pillow/Qt combinations – convert to RGB first.
img = qr.make_image(fill_color="black", back_color="white").convert("RGB")
```

`gdc_desktop/ui/screens/digital_id_screen.py`

A second change mattered as much as the conversion: the call to `generate_qr` was wrapped so that one member whose QR cannot be built does not prevent the whole screen from opening. Before, a single bad record took down the entire Digital ID feature. After, that member's card is blank and the other fifteen hundred print.

LESSON

Decide the blast radius deliberately. When one item in a list fails, the question "should this kill the list?" has a right answer, and it is usually no. Failing the whole screen is a decision, even when it is made by omission.

What the two cases share

Nothing, in their mechanisms. But both were in code paths that no automated test touched, in an application with four unit tests across 57,700 lines. Both were found by a user clicking a button.

That is the real finding, and it recurs throughout this book: the project's defect rate was not unusual. Its *detection* rate was. Faults were discovered in the order that users happened to encounter them, which is the most expensive possible order.

CHAPTER 6

The crash that was not a crash

Some failures lie about their own nature. This one presented as a crashed process, was reported as a lost connection, and was in fact a machine running out of memory.

Symptom. Building the Android application failed:

```
Daemon compilation failed: Connection to the Kotlin daemon has been
unexpectedly lost. This might be caused by the daemon being killed by
another process or the operating system, or by JVM crash.
```

```
org.jetbrains.kotlin.gradle.tasks.DaemonCrashedException
```

What the message suggests. Read plainly, it proposes three causes: another process killed the daemon, the operating system killed it, or the JVM crashed. All three sound like environmental misfortune – antivirus interference, a flaky machine, a corrupted installation. None sounds like "configuration".

This framing is actively misleading, and it is worth pausing on why. The Gradle process is not the process that died. It observed a socket close and reported what it could observe. The dying process had no opportunity to explain itself, because a JVM terminated by the operating system's memory manager does not get to write a message. So the only account available comes from a witness who saw nothing but the silence.

Mechanism. The project configured the Kotlin compiler daemon with a one-gigabyte heap:

```
org.gradle.jvmargs=-Xmx1536m -XX:MaxMetaspaceSize=512m
kotlin.daemon.jvmargs=-Xmx1024m
```

One gigabyte is generous for a small module and inadequate for this one. The shared module compiles Jetpack Compose, Hilt with its annotation processors, SQLDelight's generated sources and Apache POI in a single unit. The compiler exhausted its heap, the JVM died, the socket closed, and Gradle reported a lost connection.

There is a second resource that fails the same way and is easy to overlook. Metaspace holds class metadata, and annotation processors load an enormous number of classes. A daemon can exhaust metaspace while heap sits comfortably under its limit, producing the identical symptom. The original configuration set no metaspace limit for the daemon at all.

Fix.

```
org.gradle.jvmargs=-Xmx2560m -XX:MaxMetaspaceSize=768m -XX:+UseParallelGC
kotlin.daemon.jvmargs=-Xmx2048m -XX:MaxMetaspaceSize=512m -XX:+UseParallelGC
```

And an operational detail that wastes an afternoon when missed: **a running daemon keeps the arguments it started with**. Editing the configuration changes nothing until the existing daemon is stopped. `gradlew --stop` is not housekeeping; it is the step that makes the fix take effect.

When more memory is not available

Raising the limits got the build further – to a genuine compilation error rather than a dead socket – but it failed again on the same machine, because that machine was also running an IDE holding two gigabytes of its own. Two JVMs wanting 2.5 GB and 2 GB on a machine that did not have them simply moved the problem.

The second fix removes the second process entirely:

```
kotlin.compiler.execution.strategy=in-process
```

The compiler now runs inside the Gradle daemon. Cold builds are slower, because the compiler is no longer kept warm between runs. In exchange, there is no separate process to lose a connection to – one process cannot lose a connection to itself. For a build that otherwise cannot finish, that is an easy trade.

LESSON

When an error message describes a relationship failing – a lost connection, a closed pipe, a timeout – ask what happened to the thing at the other end. The reporter is rarely the casualty, and the casualty rarely gets to file a report.

A note on believing build errors

The same build later failed with `Compilation error. See log for more details` and no `e: line naming a file` – which is what Kotlin emits when the daemon dies mid-compilation. It looked like a code error and was another instance of the same memory problem wearing a different costume.

The discriminator is simple and worth memorising: **a real Kotlin compilation error always names a file and a line**. A "compilation error" with nothing to point at is not a compilation error.

PART III

Failures That Never Crash

The expensive ones. No dialog, no traceback, no log entry – just a system quietly doing the wrong thing.

CHAPTER 9

The cost of catching everything

A count, to begin with. Across the four applications there are 357 broad exception handlers: 180 in Python, 149 in Kotlin, 28 in TypeScript. That is roughly one for every 160 lines of code.

Each one was written for a good reason. A network call might fail; a malformed record might not parse; a file might be missing. Catching the exception keeps the application running instead of dropping the librarian back to a blank screen. The applications feel stable, and that stability is real.

But an exception handler is a decision about what should happen next, and the overwhelming majority of these made the same decision:

```
try:
    do_the_thing()
except Exception:
    pass
```

Which says: *if this fails, proceed as though it succeeded.*

That is occasionally correct. Failing to write an optional cache entry should not stop a sale. But applied broadly it converts a system that fails loudly into one that fails silently, and silent failure in a data system is not a smaller problem than a crash – it is a larger one, because it is discovered later and by someone with less information.

The shape of the damage

Consider a sync worker that pushes pending records to the cloud. Wrapped in a broad handler, a failure at any point produces: no crash, no dialog, no log entry the user would see, and a status badge that continues to read *Synced* because the code that sets it was never reached by the failure path.

The librarian catalogues books for three weeks. Every record is saved locally and every record is correct. None of them reach the network. The failure surfaces when a second device shows a different catalogue, or when the directorate's totals do not match, or – worst – when the machine dies and the records that were supposedly in the cloud turn out not to be.

Three weeks of invisible divergence is a far worse outcome than an error dialog on the first day.

What to do instead

The instruction "never catch broad exceptions" is not useful advice for a real codebase; there are places where continuing is genuinely right. Three rules proved more workable.

Catch narrowly where you can name the failure. `except ConnectionError` states a belief about what can go wrong and lets everything else through. `except Exception` states no belief at all.

Never swallow silently. Even when continuing is correct, a logged line costs nothing and is the difference between a diagnosable system and an opaque one. Most of the 357 handlers would have been acceptable with a single `logger.warning` inside them.

Distinguish "failed" from "not attempted." A sync that did not run and a sync that ran and failed should never present identically. This one distinction, made explicit in the status model, resolved a fault that had consumed days – the subject of Chapter 11.

The compensating control

Retrofitting 357 handlers is not realistic. What is realistic is an external check that does not depend on any of them being correct: compare local record counts against the cloud, per collection, and report divergence.

```
[ ok ] In sync: books      4,812 both sides
[ FAIL ] Divergence: members  local 1,504 vs cloud 1,171 (+333)
        - records are not reaching the network
```

This asks the only question that matters and does not care why the answer is wrong. It catches a failure in any of the 357 handlers, plus failures in code that has no handler at all, plus failures nobody anticipated.

LESSON

When you cannot trust a system to report its own failures, measure its outcomes from outside. An independent check on the result is worth more than any number of internal assertions about the process.

CHAPTER 10

The delete that came back

A librarian deletes a book on the Android app. It disappears. The next day it is back. This chapter is about why, and about a general principle of distributed systems that the bug illustrates exactly.

Symptom. Deletions performed on Android did not persist. Deletions performed on the desktop did. The book reappeared after a delay – usually overnight, sometimes within minutes – which made it look intermittent.

The false trail. Intermittency suggests a race condition, and a good deal of time went into examining sync timing. The behaviour was in fact perfectly deterministic; the *delay* was variable because it depended on when the next pull happened, but the outcome never varied at all.

Mechanism

The Android delete executed a hard SQL delete:

```
DELETE FROM books WHERE id = :id;
```

The row was gone from the device. Now consider how the push engine decides what to upload:

```
getUnsyncedBooks:  
SELECT * FROM books WHERE syncStatus = 'pending' AND deleted = 0;
```

It uploads rows. A deleted row is not a row. There was nothing left to tell the cloud about, so the cloud never learned of the deletion – and on the next pull, helpfully returned the book that the device appeared to be missing.

The system was working exactly as designed. The design was wrong.

The principle

In a system that synchronises by exchanging records, **a deletion is a fact that must itself be transmitted.** Absence cannot be transmitted; only presence can. If the only record of a deletion is the absence of a row, that deletion exists on one device and nowhere else, and any device that still has the row is correct to reinstate it.

The standard answer is a tombstone: instead of removing the row, mark it dead and let the marker replicate like any other change.

The fix, and the three things it required

Switching to a soft delete was the obvious part. Two further changes were needed, and each was its own small bug.

The tombstone must be eligible for pushing. The query above excluded `deleted = 0` – which was written to avoid uploading junk, and which guaranteed a tombstone could never be uploaded. It had to become:

```
-- Deleted rows MUST be included here: a soft delete is pushed to Firestore
-- as a tombstone (deleted = 1), and excluding them meant a delete never left
-- the device and the book reappeared on the next pull.
getUnsyncedBooks:
SELECT * FROM books WHERE syncStatus = 'pending';
```

The tombstone must be queued. The soft-delete statement marked the row dead but left `syncStatus` as it was. A row already marked `synced` stayed invisible to the push engine forever:

```
softDelete:
UPDATE books SET deleted = 1, syncStatus = 'pending', lastUpdated = :lastUpdated
WHERE id = :id;
```

Everything that reads must now exclude the dead. This is the cost of tombstones: every list, count, search and lookup has to filter them, and each place you forget is a deleted record reappearing in one particular view. Eleven queries needed the filter – `getAllBooks`, `getAvailableBooks`, `searchBooks`, `getBooksByCategory`, `getCategories`, `getTotalCount`, `getAvailableCount`, and the member equivalents.

One of them mattered more than the rest:

```
-- deleted = 0 matters here: without it a removed member could still sign in
-- to the OPAC portal with their old ID and PIN.
loginOpacStudent:
SELECT * FROM members WHERE memberId = :memberId AND pin = :pin AND deleted = 0
LIMIT 1;
```

A removed member could still authenticate. The deletion bug had quietly become an access-control bug – and would not have been found by testing deletion, because nobody thinks to test whether a deleted user can still log in.

Where tombstones are deliberately not applied

Two categories of query were left unfiltered, on purpose, and the reasoning deserves recording because "filter everywhere" is the wrong instinct.

The `*Static` variants used for backups keep full fidelity: a backup that silently drops deleted records is not a backup of the database. And lookups by id or sync id stay unfiltered because the sync engine itself needs to find tombstones in order to process them – filtering there would break the very mechanism the tombstones

exist for.

LESSON

Deleting data in a synchronised system is not the removal of a row; it is the creation of a record that says the row is gone. Systems that treat deletion as absence will always resurrect what they delete.

A closing observation

This bug existed only on Android. The desktop app had implemented soft deletes from the start. That asymmetry is characteristic of multi-platform systems: the same feature, implemented independently three times, will be correct in some implementations and wrong in others, and the ones that are wrong will be the ones the developer uses least.

Which is the subject of Part IV, where the same asymmetry produces something considerably worse than a reappearing library book.

CHAPTER 11

The status badge that blamed the network

For several days the working theory was that the college had a connectivity problem. It did not. The application was reporting a fault it had no evidence for, and everyone – including the people who wrote it – believed the report.

Symptom. The Android app showed a red badge reading *Offline*. Nothing synchronised. The device had a perfectly good internet connection; other applications on the same phone worked normally.

The false trail. A badge that says Offline is a strong claim, and it was taken at face value. Time went into network diagnosis: signal strength, captive portals, firewall rules, whether Firestore's endpoints were reachable from that network. The user eventually said, with some exasperation, that there was no offline issue and the real issue was sync – which was correct, and which the application had been actively contradicting.

Mechanism

The sync engine tracked its state in a single value initialised at construction:

```
private val _status = MutableStateFlow<SyncStatus>(SyncStatus.Offline)
```

Every entry point into the engine began with the same guard:

```
override fun startRealtimeSync(scope: CoroutineScope) {  
    if (currentInstitutionId.isEmpty()) return  
    ...  
}  
  
override suspend fun pushChanges() {  
    if (currentInstitutionId.isEmpty()) return  
    ...  
}
```

Read those two fragments together. If the device has no institution – no college to synchronise with – every entry point returns immediately *without touching the status*. The status therefore remains whatever it was initialised to, which is `Offline`.

So the badge was not reporting a measurement. It was reporting a default that nothing had ever overwritten. The application had never attempted a network operation, had no information about connectivity whatsoever, and was displaying a confident red assertion about the network.

Why the device had no institution

The institution id is written to preferences at login and nowhere else. The device in question had been reinstalled over an existing install, which preserved application data including a stale cached id – or, in other cases, had signed in before the profile carried an institution at all. Either way the engine had nothing to work with, and the only honest thing to display was *this device is not attached to a college*.

The fix

The state model was missing a state. Adding it took three lines:

```
/**
 * No institution is linked on this device, so there is nothing to sync
 * with. Kept separate from [Offline] because the two need opposite
 * responses – Offline resolves itself when the network returns, this one
 * never does until the user signs in again – and reporting it as Offline
 * sent people hunting for a network fault that did not exist.
 */
data object NotLinked : SyncStatus
```

It became the initial value, and was set explicitly at each early return. The badge now reads *"Not linked – sign in again"*, which is not merely more accurate: it is the actual remedy, stated on the screen where the problem appears.

The general fault

This is a state-modelling error, and it has a recognisable signature. The engine had three real conditions – *working*, *network unavailable*, and *nothing configured to work with* – and a model with only the first two. When a model lacks a state, the condition does not disappear; it gets misfiled under whichever existing state is nearest, and the user is told something false.

The two conditions that got conflated require opposite responses, which is what made the conflation so costly. *Offline* resolves itself: wait, move nearer the router, and it clears. *NotLinked* never resolves on its own, no matter how long anyone waits, because the missing thing is a login rather than a signal.

LESSON

A default value displayed as a measurement is a lie the system tells with complete confidence. Initialise state to unknown, never to a plausible answer – a wrong diagnosis on screen is worse than no diagnosis, because people act on it.

One further detail deserves recording. A related state, also displayed as *Offline*, meant that Firebase had failed to initialise at all – a build problem, typically a missing or mismatched `google-services.json`. That is a third distinct condition with a third distinct remedy, and it too was being reported as a network fault. The troubleshooting table now separates all three, because the word "Offline" had been doing the work of three different diagnoses.

CHAPTER 12

Phantoms: deleting what is already gone

This chapter concerns a bug that reported success. The wipe tool said it had deleted everything. The console agreed that the documents were gone. Several thousand records remained in storage, and were still being billed.

Symptom. The user deleted every document in Firestore by hand through the Firebase console. Afterwards, three collections – `users`, `institutions`, `colleges` – still appeared in the console and could not be removed. Running the project's own wipe tool reported nothing left to delete.

Two facts about Firestore that together produce the trap

First: a collection is not an object. It has no existence of its own. A collection appears in the console exactly as long as something lives beneath it, and vanishes when the last document goes. So a collection that refuses to disappear is not being stubborn – it is telling you, correctly, that something is still down there.

Second: deleting a document does not delete its subcollections. This is documented, and it still surprises nearly everyone. Delete `institutions/GDC-MARDAN-01` and its `books`, `members` and `issued_books` subcollections are untouched. What remains is an id that is no longer a document but still has children – a **phantom**. The console shows phantom ids in italics, which is a detail almost nobody knows to look for.

In this project each college held roughly thirty thousand documents in subcollections. Deleting the parent removed one document and orphaned thirty thousand – invisible in the console, still stored, still counted against the bill, and still able to reappear if anything recreated the parent.

The second bug, hiding behind the first

The project already had a recursive delete written precisely because of this behaviour. It did not work after a console deletion, and the reason is a single method call.

```
docs = list(col_ref.limit(batch_size).stream())
if not docs:
    return deleted
```

`stream()` executes a query and returns documents. A phantom is not a document – it has no fields, no data, nothing to return. It is invisible to any query. So `stream()` returned an empty list, the function concluded the collection was empty, and returned zero.

The tool was not failing. It was correctly reporting that there were no *documents*, in a situation where the problem was thirty thousand documents beneath something that was not a document.

The fix is a different enumeration method:

```
# list_documents(), NOT stream(). stream() returns only documents that
# exist; deleting a parent in the Firebase console leaves a PHANTOM – an
# id that is no longer a document but still holds subcollections. stream()
# cannot see phantoms, so a wipe run after a console deletion reported
# "0 documents" while every book under that college was still stored,
# still billed, and still able to reappear.
refs = list(col_ref.list_documents(page_size=batch_size))
```

`list_documents()` returns *references* rather than data, and a phantom has a reference. Having reached it, the code recurses into its subcollections, deletes the children, and then deletes the parent only if it actually exists – a phantom has nothing left to delete once its children are gone.

Why this class of bug is dangerous

A tool that fails loudly gets fixed. A tool that reports success while doing nothing produces confident wrong belief, and the belief persists until something else contradicts it. Here the contradiction came from the console, which kept showing collections that were supposed to be gone – and even then the first interpretation was that the console was wrong.

Note also that both the console route and the scripted route failed, in different ways, for the same underlying reason. Having two methods did not provide redundancy, because both rested on the same incorrect assumption: that deleting a parent deals with its children.

LESSON

Verify destructive operations by their outcome, never by their report. "Deleted 0 documents" and "there was nothing to delete" are different claims, and a tool that cannot distinguish them will tell you the wrong one with confidence.

The practical consequence was a change in procedure. Every wipe is now followed by an independent check that asserts emptiness and fails if anything survived – the same principle as Chapter 9's divergence check, applied to deletion. The tool that performs an operation is the worst possible judge of whether it worked.

CHAPTER 13

Last-write-wins and the clock you do not control

This chapter describes a weakness that has not been fixed. It is included because a casebook of only solved problems misrepresents engineering, and because knowing where your system is fragile is worth more than pretending it is not.

When two devices edit the same record while both are offline, and both later reconnect, something has to decide which version survives. NEXLIB uses last-write-wins on a timestamp: each record carries

`lastUpdated`, and the higher value wins.

This is the simplest workable strategy, and for a library system it is mostly the right one. Two librarians rarely edit the same book at the same moment, and when they do, the later edit is usually the one intended. It requires no coordination, no version vectors, and no user-facing conflict dialog that nobody would understand.

The first weakness: whose clock?

`lastUpdated` is set by the device making the change, from that device's own clock. Consider a tablet whose clock is ten minutes fast – not an exotic scenario; it is the default outcome of a dead battery, a manual timezone change, or a cheap device with no network time.

That tablet now wins every conflict it participates in, permanently and invisibly. A correction made on the desktop at 10:05 loses to an older edit made on the tablet at 09:58, because the tablet stamped it 10:08. Nobody is told. The desktop's change simply disappears on the next pull, and the librarian who made it concludes the app did not save.

Clock skew converts a fair-ish arbitration rule into a fixed hierarchy determined by whose clock runs fastest. In a deployment across hundreds of colleges with uncontrolled hardware, some device somewhere has a badly wrong clock. It is a certainty, not a risk.

The second weakness: documents, not fields

The comparison is per record, not per field. Librarian A corrects a book's ISBN; librarian B, offline, updates its shelf location. Both push. The later write replaces the whole document, and one of the two corrections is gone – even though they touched different fields and did not actually conflict.

The data loss here is silent and, to the person who made the lost edit, looks exactly like the application failing to save.

What would fix it

Three directions, in increasing order of cost.

Server timestamps. Firestore can stamp a write with the server's clock rather than the client's. This removes skew entirely for records written while online. It does not help a record created offline, which by definition has no server contact at the moment of writing – but it narrows the exposure considerably.

Field-level merging. Comparing per field rather than per document would let A's ISBN and B's shelf location both survive. This is more work, and needs care: some field groups are only meaningful together, and merging them independently produces a record that is internally inconsistent.

Detect and surface. Cheapest of the three and possibly the most valuable: when a pull overwrites a local record that had unpushed changes, record it. The project already has a `sync_conflicts` table and writes to it. Nothing ever reads it. A conflict that is logged but never surfaced is not much better than one that is not logged at all.

Why it has not been fixed

Because two blockers outrank it – an administrative credential shipping inside the desktop application, and a sync design that would cost the department seven million rupees a year at provincial scale. Both are certain harms. This is a probable one, and the pilot exists partly to find out how probable.

That is a legitimate engineering decision, and it becomes an illegitimate one the moment it stops being written down. An unfixed weakness that is documented, scheduled and disclosed is a managed risk. The same weakness unmentioned is a defect waiting to be discovered by a librarian who will be told the app is working correctly.

LESSON

Write down the faults you are choosing not to fix, and say why. The knowledge that a system is fragile in a specific place is itself a deliverable – and it is the first thing lost when the person who knew it moves on.

PART IV

Security Failures

Where a bug stops being an inconvenience and becomes a disclosure. Three of these shipped; one is still in the code as this is written.

CHAPTER 14

One rule, every college's data

The most serious defect in this book was not found by a crash, a test, or a user complaint. It was found by reading a list of role names and noticing that one of them meant two different things.

The context. NEXLIB is multi-tenant. Each college's books, members and loans live under `/institutions/{collegeId}/` and must be readable only by that college. Above them sits a registry, `/directorate_index`, holding one small summary document per college – counts and a heartbeat, no patron records – which the Higher Education Department reads to see the province.

The registry is deliberately aggregate-only, and that design is sound: a document containing nothing but counts is safe to expose to oversight staff in a way that patron records never would be. The failure was in deciding who counted as oversight staff.

The mechanism: one word, two meanings

The system has a per-college role named `director`. In the desktop application's own code it is documented as an alias for `college_admin` – the person who runs one college's library. It is an ordinary account that hundreds of colleges would hold.

It also reads like the word for the Directorate. And so it appeared in the list of roles granted network-wide oversight, in two places at once – the web application's role constants, and the security rules themselves:

```
function isDirectorateAdmin() {
  return userRole() in [
    'DirectorateAdmin', 'directorate_admin',
    'Director', 'director',          // ← a PER-COLLEGE role
    'owner', 'admin', 'college_admin' // ← also per-college
  ];
}
```

On its own that would have been a client-side annoyance. Combined with the rule governing the registry, it was a disclosure:

```
match /directorate_index/{collegeId} {
  allow read: if signedIn();
}
```

Any signed-in user could read the entire registry. Every college's book counts, member counts, active loans, overdue figures and outstanding fines – readable by any librarian at any of the other colleges, and by any account that could sign in at all.

The client-side gate made it worse rather than better, because it created the appearance of a control. The web application checked the role list before showing the directorate dashboard – which meant a college's own owner or director account simply *saw the menu item*, clicked it, and got the province. No circumvention required. And even for accounts the client did block, the block was cosmetic: the rules permitted the read, so anyone willing to call the API directly had the data regardless.

The fix

Two changes, and the order of importance matters.

```
// Directorate-level oversight: 'director'/'Director' is a PER-COLLEGE role
// (isDirector() treats it the same as owner/admin/college_admin), so it must
// never appear here: including it previously let every ordinary college
// account read every other college's aggregate counts network-wide.
function isDirectorateAdmin() {
  return userRole() in ['DirectorateAdmin', 'directorate_admin'];
}

match /directorate_index/{collegeId} {
  allow read: if isDirectorateAdmin() || belongsToCollege(collegeId);
}
```

The second rule carries a judgement worth stating. A college may read its own published summary – it needs to, in order to publish it. But it may not read anyone else's, and the registry is not left open to "any signed-in user" the way some other collections are. A college's own counts are not another college's business, even though they contain no patron data.

The client-side role list was fixed too, but as tidiness rather than as the remedy. **The rules are the control; the client is a convenience.** Any security check that exists only in an application the user controls is not a security check.

The part that should worry you

The project had a security test suite – forty-one behavioural tests running against the Firestore emulator, written specifically to verify tenant isolation. The tests passed throughout.

They passed because one of them asserted the broken behaviour was correct. The suite had been written by examining what the rules did and encoding it, which is the natural way to write tests for an existing system and is precisely how a test suite becomes a record of current behaviour rather than a check on intended behaviour. Once a defect is written into a test, the suite stops being able to find it. It actively defends it: fix the bug and the tests go red, which looks like a regression.

Fixing this required editing the test that encoded the fault, then adding coverage for what should have been true all along:

```
await check("plain college director CANNOT read another college's snapshot", () =>
  assertFails(getDoc(doc(director, "directorate_index/GDC-ZIAM"))));
await check("plain college director CANNOT list the whole registry", () =>
  assertFails(getDocs(collection(director, "directorate_index"))));
```

The seeded fixture now includes a per-college `director` account for no purpose other than proving it cannot see across tenants – the exact account that the original list had quietly promoted.

LESSON

Tests written from the implementation certify the implementation. Security tests in particular must be written from the requirement – who must not be able to do what – or they will pass whatever the code happens to do, including the thing you are most afraid of.

Why it survived so long

The asymmetry from Chapter 10 again, in its most damaging form. The desktop application reaches Firestore through an administrative credential that bypasses security rules entirely. It can read everything by design, so no rules defect is observable from it – and the desktop app was the one used daily during development. A category of fault had been made invisible on the platform where it would have been noticed first.

That is not a testing failure. It is an architectural decision, made early for convenience, whose cost arrived much later and in a form nobody connected back to it.

END OF THIS EDITION

Chapters 1–3, 7–8, 15–26 in preparation

Each follows the same structure: symptom, false trails, mechanism, fix, lesson – drawn from the project's own commit history, where every case in this book is recorded with the reasoning that produced it.